

Quotient inductive-inductive types in the setoid model*

Ambrus Kaposi and Szumi Xie

Eötvös Loránd University, Budapest, Hungary
akaposi@inf.elte.hu and szumixie@gmail.com

Introduction. The setoid model of type theory provides a way to bootstrap functional extensionality [1] and propositional extensionality [3]: the setoid model can be defined in an intensional metatheory with a universe of definitionally proof irrelevant (strict) propositions **SProp**. It is a strict model in such a metatheory, that is, all equalities of the model (e.g. β and η for function space) hold definitionally. As a result, we obtain a model construction: any model of type theory with **SProp** can be turned into another model, its “setoidified” version which supports these extra principles. In addition to functional and propositional extensionality, the setoid model justifies propositional truncation¹, quotient types² and countable choice³.

Since Agda supports **SProp** [9], it is a convenient tool to experiment with the setoid model. It is straightforward to formalise the setoid model as a category with families (CwF [6]) with Π , Σ , unit, empty, **Bool**, $\mathbb{N}$, **Id** types, a universe of strict propositions. Extending the setoid model with a (non-univalent) universe of sets is harder, it was shown by Altenkirch et al. [2] that it can be done using a special form of induction-recursion or large induction-induction or an **SProp**-valued identity type with transport over types.

Until recently we thought [12] that general inductive types and even quotient inductive-inductive types (QIITs, initial algebras of generalised algebraic theories [13, 5]) are unproblematic in the setoid model, provided we have (possibly **SProp**-sorted) inductive-inductive types in the metatheory. Simon Boulier pointed out that our formalisations of Martin-Löf’s identity type⁴ and the universal QIIT⁵ only provide eliminators in the empty context. They can be salvaged using a method related to the local universes construction [14] which we explain below.

The setoid model. A context or a closed type in this model is a setoid, i.e. a set (we say set instead of (Agda) type to avoid confusion) together with an **SProp**-valued equivalence relation. A type over a context $\Gamma = (|\Gamma|, \sim_\Gamma)$ is a displayed setoid with a fibration condition $\text{coe}_A : x \sim_\Gamma x' \rightarrow |A| x \rightarrow |A| x'$ such that $x \sim_A (\text{coe}_A p x)$. Substitutions (and terms) are (dependent) functions between the underlying sets which preserve the relations.

Example: Con-Ty. To illustrate the general method, we explain how to construct the following QIIT in the setoid model⁶. It has two sorts, five constructors and one equality constructor.

Con : Set	U : Ty γ
Ty : Con $\rightarrow$ Set	El : Ty ($\gamma \triangleright \mathbf{U}$)
• : Con	Σ : (a : Ty γ) $\rightarrow$ Ty ($\gamma \triangleright a$) $\rightarrow$ Ty γ
$- \triangleright -$: (γ : Con) $\rightarrow$ Ty $\gamma \rightarrow$ Con	eq : $\gamma \triangleright \Sigma a b = \gamma \triangleright a \triangleright b$

*The first author was supported by the ÚNKP-20-5 New National Excellence Program of the Ministry for Innovation and Technology and by the Bolyai Fellowship of the Hungarian Academy of Sciences. The second author was supported by the European Union, co-financed by the European Social Fund (EFOP-16-2017-00002).

¹<https://bitbucket.org/akaposi/setoid/src/master/agda/Model/Trunc.agda>

²<https://bitbucket.org/akaposi/setoid/src/master/agda/Model/Quotient.agda>

³<https://bitbucket.org/akaposi/setoid/src/master/agda/Model/CountableChoice.agda>

⁴<https://bitbucket.org/akaposi/qiit/src/master/Setoid/Path.agda>

⁵<https://bitbucket.org/akaposi/qiit/src/master/Setoid/UniversalQIIT/>

⁶<https://bitbucket.org/akaposi/qiit/src/master/Setoid/ConTy2.agda>

We omitted some arguments, e.g. $\mathbf{U}$ implicitly takes a parameter γ . In the setoid model, we need to define a type $\mathbf{Con}$ in the empty context, a type $\mathbf{Ty}$ over $\mathbf{Con}$, and their elimination principles. We start by defining in Agda an inductive-inductive type (IIT) with these sorts:

$$\begin{array}{ll} |\mathbf{Con}| : \mathbf{Set} & |\mathbf{Ty}| : |\mathbf{Con}| \rightarrow \mathbf{Set} \\ \sim_{\mathbf{Con}} : |\mathbf{Con}| \rightarrow |\mathbf{Con}| \rightarrow \mathbf{SProp} & \sim_{\mathbf{Ty}} : \gamma \sim_{\mathbf{Con}} \gamma' \rightarrow |\mathbf{Ty}| \gamma \rightarrow |\mathbf{Ty}| \gamma' \rightarrow \mathbf{SProp} \end{array}$$

The constructors of $|\mathbf{Con}|$ are $|\bullet|$ and $|\triangleright|$, the constructors of $|\mathbf{Ty}|$ are $|\mathbf{U}|$, $|\mathbf{E}|$ and $|\Sigma|$, while $\sim_{\mathbf{Con}}$ has a constructor $|\mathbf{eq}|$. In addition, $\sim_{\mathbf{Con}}$ and $\sim_{\mathbf{Ty}}$ have constructors stating that it is an equivalence relation, and they have congruence constructors for each point constructor, e.g. there is $\sim_{\triangleright} : (p : \gamma \sim_{\mathbf{Con}} \gamma') \rightarrow \sim_{\mathbf{Ty}} p a a' \rightarrow (\gamma \triangleright a) \sim_{\mathbf{Con}} (\gamma' \triangleright a')$. Finally, $|\mathbf{Ty}|$ has a constructor $\mathbf{coe}_{\mathbf{Ty}} : \gamma_0 \sim_{\mathbf{Con}} \gamma_1 \rightarrow |\mathbf{Ty}| \gamma_0 \rightarrow |\mathbf{Ty}| \gamma_1$ and $\sim_{\mathbf{Ty}}$ a constructor for $\sim_{\mathbf{Ty}} p a (\mathbf{coe}_{\mathbf{Ty}} p a)$. Thus the IIT is the “fibrant equivalence congruence closure” of the constructors.

With the aid of this IIT (note that it has both $\mathbf{Set}$ and $\mathbf{SProp}$ -sorts) we define the type formation rules and constructors of the Con-Ty QIIT in the setoid model *in the empty context*: the underlying set for $\mathbf{Con}$ is $|\mathbf{Con}|$, the relation is $\sim_{\mathbf{Con}}$, the witnesses for the equivalence relation come from the corresponding constructors of $\sim_{\mathbf{Con}}$, and so on. Thus $\mathbf{Con}$ becomes a type in the empty context in the setoid model. $\mathbf{Ty}$ is a type over the one-element context $\mathbf{Con}$. $\bullet$ is a term in the empty context of type $\mathbf{Con}$, and so on. We added exactly the required structure to the IIT to be able to define the constructors. The $\mathbf{eq}$ equality constructor is given by $|\mathbf{eq}|$. Given a Con-Ty algebra in the empty context, we define four functions by recursion-recursion as a first step towards the (non-dependent) elimination principle.

The type formation rules and constructors can easily be lifted from the empty context to an arbitrary context and all the substitution laws hold definitionally. We also need that for any context Γ , we can eliminate into a Con-Ty algebra in Γ . Our setoid model has Π types and $\mathbf{K}$ constant types (a context can be turned into a type). With the help of these we can turn a type C in Γ into the type $\Pi(x : \mathbf{K} \Gamma).C[x]$ which is in the empty context. This way we turn the algebra in Γ into an algebra in the empty context on which we can apply our previously defined elimination principle. This way we obtain the eliminator in arbitrary contexts. All computation rules of this eliminator are definitional.

We prove uniqueness of the eliminator by induction-induction on $|\mathbf{Con}|$ and $|\mathbf{Ty}|$. The substitution law of the eliminator is proven by another induction-induction on the same sets.

In our formalisation, Con-Ty has an additional infinitary constructor (an infinitary Π type indexed by a code of a setoid in a universe). It seems that with the help of a universe in the setoid model, open QIITs and those with infinitary constructors can be handled as well. Note that in contrast with the unordered infinitely branching tree example in [4], we do not use the (countable) axiom of choice to construct this QIIT.

Arbitrary QIITs Signatures for QIITs can be specified using the theory of QIIT signatures [13] (ToS) which is itself an infinitary QIIT. We formalised⁷ that the setoid model supports ToS in the empty context. Based on our experience with Con-Ty, we expect that it is possible to lift ToS from the empty context to arbitrary contexts. If this succeeds, then following [13], we can construct all QIITs from ToS with propositional computation rules. As the construction of [13] is performed in extensional type theory, we use Hofmann’s conservativity result [10, 15] to transfer it to the setoid model. This way however we only obtain propositional computation rules. It remains to be proven that all QIITs are supported by the setoid model with definitional computation rules. We plan to do this by induction on QIIT signatures.

We would also like to understand the relationship of this construction to that of higher inductive types (HITs) in cubical models [8, 7] and how they could be extended to HIITs [11].

⁷<https://bitbucket.org/akaposi/qiit/src/master/Setoid2/ToS/>

References

- [1] Thorsten Altenkirch. Extensional equality in intensional type theory. In *14th Symposium on Logic in Computer Science*, pages 412 – 420, 1999.
- [2] Thorsten Altenkirch, Simon Boulier, Ambrus Kaposi, Christian Sattler, and Filippo Sestini. Constructing a universe for the setoid model. In Stefan Kiefer and Christine Tasson, editors, *Foundations of Software Science and Computation Structures - 24th International Conference, FOSSACS 2021, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2021, Luxembourg City, Luxembourg, March 27 - April 1, 2021, Proceedings*, volume 12650 of *Lecture Notes in Computer Science*, pages 1–21. Springer, 2021.
- [3] Thorsten Altenkirch, Simon Boulier, Ambrus Kaposi, and Nicolas Tabareau. Setoid type theory—a syntactic translation. In Graham Hutton, editor, *Mathematics of Program Construction*, pages 155–196, Cham, 2019. Springer International Publishing.
- [4] Thorsten Altenkirch and Ambrus Kaposi. Type theory in type theory using quotient inductive types. In Rastislav Bodik and Rupak Majumdar, editors, *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2016, St. Petersburg, FL, USA, January 20 - 22, 2016*, pages 18–29. ACM, 2016.
- [5] John Cartmell. Generalised algebraic theories and contextual categories. *Annals of Pure and Applied Logic*, 32:209–243, 1986.
- [6] Simon Castellan, Pierre Clairambault, and Peter Dybjer. Categories with families: Unityped, simply typed, and dependently typed. *CoRR*, abs/1904.00827, 2019.
- [7] Evan Cavallo and Robert Harper. Higher inductive types in cubical computational type theory. *Proc. ACM Program. Lang.*, 3(POPL), January 2019.
- [8] Thierry Coquand, Simon Huber, and Anders Mörtberg. On higher inductive types in cubical type theory. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science, LICS ’18*, page 255–264, New York, NY, USA, 2018. Association for Computing Machinery.
- [9] Gaëtan Gilbert, Jesper Cockx, Matthieu Sozeau, and Nicolas Tabareau. Definitional Proof-Irrelevance without K. *Proceedings of the ACM on Programming Languages*, pages 1–28, January 2019.
- [10] Martin Hofmann. Conservativity of equality reflection over intensional type theory. In *TYPES 95*, pages 153–164, 1995.
- [11] Ambrus Kaposi and András Kovács. Signatures and Induction Principles for Higher Inductive-Inductive Types. *Logical Methods in Computer Science*, Volume 16, Issue 1, February 2020.
- [12] Ambrus Kaposi and Szumi Xie. A model of type theory with quotient inductive-inductive types. In Ugo de’ Liguoro and Stefano Berardi, editors, *26th International Conference on Types for Proofs and Programs, TYPES 2020*. University of Turin, 2020.
- [13] András Kovács and Ambrus Kaposi. Large and infinitary quotient inductive-inductive types. In Holger Hermanns, Lijun Zhang, Naoki Kobayashi, and Dale Miller, editors, *LICS ’20: 35th Annual ACM/IEEE Symposium on Logic in Computer Science, Saarbrücken, Germany, July 8-11, 2020*, pages 648–661. ACM, 2020.
- [14] Peter Lefanu Lumsdaine and Michael A. Warren. The local universes model: An overlooked coherence construction for dependent type theories. *ACM Trans. Comput. Logic*, 16(3), July 2015.
- [15] Théo Winterhalter, Matthieu Sozeau, and Nicolas Tabareau. Eliminating reflection from type theory. In Assia Mahboubi and Magnus O. Myreen, editors, *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2019, Cascais, Portugal, January 14-15, 2019*, pages 91–103. ACM, 2019.